Assignment-11.4

K . Jashwanth

2303A52300

Batch 42

Task-1: Stack Implementation for Undo Operations (LIFO)

PROMPT:

Create a Python UndoStack class that implements a stack for undo operations. Use a private list to store actions. Include push() to add actions, pop() to remove and return the most recent (return None if empty), peek() to view without removing (return None if empty), is_empty() to check emptiness, size() to return the count, and __str__() for string representation. Use type hints (typing.Any, typing.Optional). Add a test function that exercises all methods, including edge cases like empty stacks. Handle empty stack cases without exceptions.

CODE:

```python
#Create a Python UndoStack class that implements a stack for undo operations. Use a private list to store ac
from typing import Any, Optional
class UndoStack:
    """Stack implementation for undo functionality."""
    def __init__(self):
        self._items = []
    def push(self, action: Any) -> None:
        """Add an action to the top of the stack."""
        self._items.append(action)
    def pop(self) -> Optional[Any]:
        """Remove and return the most recent action."""
        return self._items.pop() if not self.is_empty() else None
    def peek(self) -> Optional[Any]:
        """Return the most recent action without removing it."""
        return self._items[-1] if not self.is_empty() else None
    def is_empty(self) -> bool:
        """Check if the stack is empty."""
        return len(self._items) == 0
    def size(self) -> int:
        """Return the number of actions in the stack."""
        return len(self._items)
    def __str__(self) -> str:
        return str(self._items)
def test_undo_stack():
    """Test the UndoStack class."""
    print("TESTING UndoStack CLASS")
    print("=" * 100)
    stack = UndoStack()
    # Test 1: Pushing actions onto the stack
    print("\n[Test 1] Pushing actions onto the stack")
    stack.push("Action 1")
    stack.push("Action 2")
    stack.push("Action 3")
```

```python
def test_undo_stack():
    print(f"Current stack: {stack}")
    # Test 2: Peeking at the top action
    print("\n[Test 2] Peeking at the top action")
    top_action = stack.peek()
    expected = "Action 3"
    print(f"Top action: {top_action}, Expected: {expected}")
    if top_action == expected:
        print("✓ PASSED")
    # Test 3: Popping actions from the stack
    print("\n[Test 3] Popping actions from the stack")
    popped1 = stack.pop()
    expected1 = "Action 3"
    print(f"Popped action: {popped1}, Expected: {expected1}")
    popped2 = stack.pop()
    expected2 = "Action 2"
    print(f"Popped action: {popped2}, Expected: {expected2}")
    # Test 4: Checking if the stack is empty
    print("\n[Test 4] Checking if the stack is empty")
    is_empty_before = stack.is_empty()
    expected_empty_before = False
    print(f"Is stack empty? {is_empty_before}, Expected: {expected_empty_before}")
    if is_empty_before == expected_empty_before:
        print("✓ PASSED")
    # Pop the remaining action
    stack.pop()
    is_empty_after = stack.is_empty()
    expected_empty_after = True
    print(f"Is stack empty after popping all actions? {is_empty_after}, Expected: {expected_empty_after}")
    if is_empty_after == expected_empty_after:
        print("✓ PASSED")
if __name__ == "__main__":
    test_undo_stack()
```

OUTPUT:

```
ve\Desktop\AI ASSISTED CODING\.venv\Scripts\python.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI AS
SISTED CODING/Lab-13.py"
TESTING UndoStack CLASS
==========

[Test 1] Pushing actions onto the stack
Current stack: ['Action 1', 'Action 2', 'Action 3']

[Test 2] Peeking at the top action
Top action: Action 3, Expected: Action 3
✓ PASSED

[Test 3] Popping actions from the stack
Popped action: Action 3, Expected: Action 3
Popped action: Action 2, Expected: Action 2

[Test 4] Checking if the stack is empty
Is stack empty? False, Expected: False
✓ PASSED
Is stack empty after popping all actions? True, Expected: True
✓ PASSED
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION:

The UndoStack class implements a stack using a private _items list. It provides push() to add actions, pop() to remove and return the top item (returns None if empty), peek() to view the top without removing it (returns None if empty), is_empty() to check if the stack is empty, size() to return the count, and __str__() for string representation. The class uses type hints (Any, Optional) and handles empty stacks without exceptions. The test_undo_stack() function exercises all methods: it pushes three actions, peeks at the top, pops items in LIFO order, and verifies is_empty() before and after emptying the stack. The implementation follows standard stack behavior where the last item added is the first one removed.

Task-2: Queue for Customer Service Requests (FIFO) List

Based Queue:

PROMPT:

Create a Python CustomerServiceQueue class implementing a FIFO queue using a private list. Include enqueue(request: Any), dequeue() -> Optional[Any] (returns None if empty), is_empty() -> bool, size() -> int, and __str__() methods. Use type hints (typing.Any, typing.Optional) and handle empty queue cases gracefully. Provide a comprehensive test function covering all methods and edge cases, including dequeuing from an empty queue.

CODE:

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
Create a Python CustomerServiceQueue class implementing a FIFO queue using a private list. Include enqueue(
from typing import Any, Optional
class CustomerServiceQueue:
    """Queue implementation for customer service requests (FIFO)."""
    def __init__(self):
        """Initialize an empty queue."""
        self._items = []
    def enqueue(self, request: Any) -> None:
        """Add a service request to the end of the queue."""
        self._items.append(request)
    def dequeue(self) -> Optional[Any]:
        """Remove and return the first service request (FIFO)."""
        return self._items.pop(0) if not self.is_empty() else None
    def is_empty(self) -> bool:
        """Check if the queue is empty."""
        return len(self._items) == 0
    def size(self) -> int:
        """Return the number of requests in the queue."""
        return len(self._items)
    def __str__(self) -> str:
        """Return string representation of the queue."""
        return str(self._items)
def test_customer_service_queue():
    """Test the CustomerServiceQueue class."""
    print("TESTING CustomerServiceQueue CLASS")
    print("=" * 100)
    queue = CustomerServiceQueue()
    # Test 1: Enqueue service requests
    print("\n[Test 1] Adding service requests to the queue")
    queue.enqueue("Request 1: Password Reset")
    queue.enqueue("Request 2: Account Inquiry")
    queue.enqueue("Request 3: Billing Question")
    print(f"Current queue: {queue}")
```

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
def test_customer_service_queue():
    print(f"Queue size: {queue.size()}")
    # Test 2: Check if queue is empty
    print("\n[Test 2] Checking if queue is empty")
    is_empty = queue.is_empty()
    print(f"Is queue empty? {is_empty}, Expected: False")
    if is_empty == False:
        print("✓ PASSED")
    # Test 3: Dequeue requests (FIFO order)
    print("\n[Test 3] Processing service requests (FIFO)")
    request1 = queue.dequeue()
    expected1 = "Request 1: Password Reset"
    print(f"Dequeued: {request1}, Expected: {expected1}")
    if request1 == expected1:
        print("✓ PASSED")
    request2 = queue.dequeue()
    expected2 = "Request 2: Account Inquiry"
    print(f"Dequeued: {request2}, Expected: {expected2}")
    if request2 == expected2:
        print("✓ PASSED")
    print(f"Remaining queue: {queue}")
    # Test 4: Dequeue remaining request
    request3 = queue.dequeue()
    expected3 = "Request 3: Billing Question"
    print(f"Dequeued: {request3}, Expected: {expected3}")
    if request3 == expected3:
        print("✓ PASSED")
    # Test 5: Check empty queue
    print("\n[Test 4] Checking empty queue")
    is_empty_after = queue.is_empty()
    print(f"Is queue empty? {is_empty_after}, Expected: True")
    if is_empty_after == True:
        print("✓ PASSED")
```

```python
def test_customer_service_queue():
    # Test 6: Dequeue from empty queue
    print("\n[Test 5] Dequeue from empty queue")
    empty_dequeue = queue.dequeue()
    print(f"Dequeued from empty queue: {empty_dequeue}, Expected: None")
    if empty_dequeue is None:
        print("✓ PASSED")
if __name__ == "__main__":
    test_customer_service_queue()
```

OUTPUT:

```
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\.venv\Scripts\python.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI ASSISTED CODING/Lab-13.py"
TESTING CustomerServiceQueue CLASS
==========

[Test 1] Adding service requests to the queue
Current queue: ['Request 1: Password Reset', 'Request 2: Account Inquiry', 'Request 3: Billing Question']
Queue size: 3

[Test 2] Checking if queue is empty
Is queue empty? False, Expected: False
✓ PASSED

[Test 3] Processing service requests (FIFO)
Dequeued: Request 1: Password Reset, Expected: Request 1: Password Reset
✓ PASSED
Dequeued: Request 2: Account Inquiry, Expected: Request 2: Account Inquiry
✓ PASSED
Remaining queue: ['Request 3: Billing Question']
Dequeued: Request 3: Billing Question, Expected: Request 3: Billing Question
✓ PASSED

[Test 4] Checking empty queue
Is queue empty? True, Expected: True
✓ PASSED

[Test 5] Dequeue from empty queue
Dequeued from empty queue: None, Expected: None
✓ PASSED
```

Deque-based optimized queue:

PROMPT:

Create a Python CustomerServiceQueue class using collections.deque for an optimized FIFO queue. Include enqueue(request: Any), dequeue() -> Optional[Any] (returns None if empty), is_empty() -> bool, size() -> int, and __str__() methods. Ensure all operations are e icient (O(1)) and handle empty queue cases gracefully. Also, provide a test function covering all methods and edge cases, with a brief explanation of why deque is more e icient than a list-based queue.

CODE:

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
#Create a Python CustomerServiceQueue class using collections.deque for an optimized FIFO queue. Include e
from collections import deque
from typing import Any, Optional
class CustomerServiceQueue:
    """
    A queue for managing customer service requests using FIFO order,
    optimized using collections.deque for efficient enqueue and dequeue operations.
    """
    def __init__(self) -> None:
        self._queue: deque[Any] = deque[Any]()  # private deque to store requests
    def enqueue(self, request: Any) -> None:
        """Add a service request to the end of the queue."""
        self._queue.append(request)
    def dequeue(self) -> Optional[Any]:
        """
        Remove and return the first service request from the front of the queue.
        Returns None if the queue is empty.
        """
        if self.is_empty():
            return None
        return self._queue.popleft()  # O(1) operation
    def is_empty(self) -> bool:
        """Check if the queue is empty."""
        return len(self._queue) == 0
    def size(self) -> int:
        """Return the number of requests in the queue."""
        return len(self._queue)
    def __str__(self) -> str:
        """Return a string representation of the queue."""
        return " <- ".join(str(req) for req in self._queue)
# ----------------- Test Function -----------------
def test_customer_service_queue():
    queue = CustomerServiceQueue()
```

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
def test_customer_service_queue():
    print("Queue empty?", queue.is_empty())
    print("Dequeue from empty queue:", queue.dequeue())
    # Enqueue requests
    queue.enqueue("Request 1")
    queue.enqueue("Request 2")
    queue.enqueue("Request 3")
    print("Queue after enqueues:", queue)
    # Dequeue requests
    print("Dequeued:", queue.dequeue())
    print("Queue now:", queue)
    # Test size and empty checks
    print("Queue size:", queue.size())
    print("Queue empty?", queue.is_empty())
    # Dequeue remaining requests
    queue.dequeue()
    queue.dequeue()
    print("Queue after removing all requests:", queue)
    print("Dequeue from empty queue:", queue.dequeue())
# Run the test
test_customer_service_queue()
```

OUTPUT:

```
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:\Users\Pranitha Rao\OneDri
ve\Desktop\AI ASSISTED CODING\.venv\Scripts\python.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI AS
SISTED CODING/Lab-13.py"
Queue empty? True
Dequeue from empty queue: None
Queue after enqueues: Request 1 <- Request 2 <- Request 3
Dequeued: Request 1
Queue now: Request 2 <- Request 3
Queue size: 2
Queue empty? False
Queue after removing all requests:
Dequeue from empty queue: None
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION:

In a customer service system, a list-based queue can be implemented using a Python list where enqueue is performed with append() and dequeue with pop(0). While adding requests is e icient (O(1)), removing requests from the front is slow (O(n)) because all remaining elements must be shifted, which can cause delays as the number of requests grows. An optimized alternative uses collections.deque, where both append() and popleft() operations run in O(1) time. This makes deque-based queues highly e icient for FIFO operations, allowing the system to handle large numbers of service requests quickly and consistently, ensuring that requests are processed in the order they arrive without performance degradation.

Task-3: Singly Linked List for Dynamic Playlist Management

PROMPT:

Create a Python implementation of a Singly Linked List for dynamic playlist management. The class should include insert_at_end(song) to add a song at the end, delete_value(song) to remove a song by value, and traverse() to display all songs in order. Add inline comments explaining pointer manipulation and highlight tricky parts such as inserting at the end, deleting the head, or handling a single-node list. Also, suggest and implement test cases for edge scenarios, including an empty list, a singlenode list, and deletion at the head, middle, and tail.

CODE:

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
#Create a Python implementation of a Singly Linked List for dynamic playlist management. The class should i
from typing import Optional
class Node:
    """A node in the singly linked list representing a song."""
    def __init__(self, data: str):
        self.data: str = data
        self.next: Optional['Node'] = None  # Pointer to the next node
class Playlist:
    """Singly linked list implementation for dynamic playlist management."""
    def __init__(self):
        self.head: Optional[Node] = None  # Start of the playlist
    def insert_at_end(self, song: str) -> None:
        """Insert a new song at the end of the playlist."""
        new_node = Node(song)
        if self.head is None:
            # List is empty, new node becomes the head
            self.head = new_node
            return
        # Traverse to the end of the list
        current = self.head
        while current.next:
            current = current.next
        # Link the last node to the new node
        current.next = new_node
    def delete_value(self, song: str) -> bool:
        """
        Delete the first occurrence of a song from the playlist.
        Returns True if deleted, False if not found.
        """
        current = self.head
        prev = None
        # Traverse the list to find the song
        while current:
```

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
class Playlist:
    def delete_value(self, song: str) -> bool:
            if current.data == song:
                if prev is None:
                    # Deleting the head node
                    self.head = current.next
                else:
                    # Skip over the current node
                    prev.next = current.next
                return True
            prev = current
            current = current.next
        return False  # Song not found
    def traverse(self) -> None:
        """Print all songs in the playlist in order."""
        current = self.head
        if not current:
            print("Playlist is empty.")
            return
        songs = []
        while current:
            songs.append(current.data)
            current = current.next
        print(" -> ".join(songs))
# ----------------- Test Function -----------------
def test_playlist():
    playlist = Playlist()
    # Edge case: traverse empty playlist
    playlist.traverse()
    # Insert songs
    playlist.insert_at_end("Song A")
    playlist.insert_at_end("Song B")
    playlist.insert_at_end("Song C")
```

Lab-13.py

C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING\Lab-13.py

```python
def test_playlist():
    playlist.traverse()
    # Delete head song
    playlist.delete_value("Song A")
    playlist.traverse()
    # Delete middle song
    playlist.delete_value("Song B")
    playlist.traverse()
    # Delete tail song
    playlist.delete_value("Song C")
    playlist.traverse()
    # Attempt to delete from empty list
    result = playlist.delete_value("Song X")
    print("Delete non-existent song:", result)
# Run test
test_playlist()
```

OUTPUT:

```
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:\Users\Pranitha Rao\OneDri
ve\Desktop\AI ASSISTED CODING\.venv\Scripts\python.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI AS
SISTED CODING/Lab-13.py"
Playlist is empty.
Song A -> Song B -> Song C
Song B -> Song C
Song C
Playlist is empty.
Delete non-existent song: False
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION:

The Playlist class uses a singly linked list to manage songs dynamically. insert_at_end adds a song at the end by traversing the list and linking the new node, while delete_value removes a song by updating pointers, handling edge cases like deleting the head, tail, or a single-node list. traverse prints all songs in order, or indicates if the playlist is empty. This structure allows e icient insertion and deletion without shifting elements, and the test cases cover empty lists, single-node lists, and deletions at di erent positions to ensure robust functionality.

Task-4: Binary Search Tree for Fast Record Lookup PROMPT:

Complete a Python Binary Search Tree (BST) implementation for a student record system, where quick searching by roll number is needed. The class should include insert(value) to add a new roll number, search(value) to check if a roll number exists, and inorder_traversal() to return all roll numbers in sorted order. Add meaningful docstrings, ensure correct BST behavior, and provide a brief explanation of how BST improves search e iciency compared to linear search, including best-case and worstcase performance.

CODE:

Lab-13.py

```python
#Complete a Python Binary Search Tree (BST) implementation for a student record system, where quick searchi
from typing import Optional, List
class Node:
    """Node of a Binary Search Tree representing a student record."""
    def __init__(self, value: int):
        self.value: int = value
        self.left: Optional['Node'] = None
        self.right: Optional['Node'] = None
class BST:
    """Binary Search Tree for fast student record lookup by roll number."""
    def __init__(self):
        self.root: Optional[Node] = None
    def insert(self, value: int) -> None:
        """Insert a value into the BST."""
        if self.root is None:
            self.root = Node(value)
            return
        self._insert_recursive(self.root, value)
    def _insert_recursive(self, node: Node, value: int) -> None:
        if value < node.value:
            if node.left is None:
                node.left = Node(value)
            else:
                self._insert_recursive(node.left, value)
        elif value > node.value:
            if node.right is None:
                node.right = Node(value)
            else:
                self._insert_recursive(node.right, value)
        # Ignore duplicates
    def search(self, value: int) -> bool:
        """Search for a value in the BST. Returns True if found, else False."""
        return self._search_recursive(self.root, value)
```

```python
class BST:
    def _search_recursive(self, node: Optional[Node], value: int) -> bool:
        if node is None:
            return False
        if value == node.value:
            return True
        elif value < node.value:
            return self._search_recursive(node.left, value)
        else:
            return self._search_recursive(node.right, value)
    def inorder_traversal(self) -> List[int]:
        """Return a list of values from the BST in sorted order."""
        result: List[int] = []
        self._inorder_recursive(self.root, result)
        return result
    def _inorder_recursive(self, node: Optional[Node], result: List[int]) -> None:
        if node:
            self._inorder_recursive(node.left, result)
            result.append(node.value)
            self._inorder_recursive(node.right, result)
# ----------------- Test Function -----------------
def test_bst():
    bst = BST()
    # Insert roll numbers
    for roll in [50, 30, 70, 20, 40, 60, 80]:
        bst.insert(roll)
    print("Inorder Traversal (sorted):", bst.inorder_traversal())
    print("Search 40:", bst.search(40))
    print("Search 90:", bst.search(90))
test_bst()
```

OUTPUT:

```
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:\Users\Pranitha Rao\OneDri
ve\Desktop\AI ASSISTED CODING\.venv\Scripts\python.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI AS
SISTED CODING/Lab-13.py"
Inorder Traversal (sorted): [20, 30, 40, 50, 60, 70, 80]
Search 40: True
Search 90: False
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION:

The BST stores student roll numbers so that each node's left child has smaller values and the right child has larger values. This structure allows search to skip half of the remaining nodes at each step, improving e iciency over linear search. In the best case, the BST is balanced, giving O(log n) search time, while in the worst case, it becomes skewed like a linked list, leading to O(n) search time. Inorder traversal returns all records in sorted order, making BST both e icient and ordered for record lookup.

Task-5: Graph Traversal for Social Network Connections

PROMPT:

Implement a Python graph using an adjacency list to model a social network where users are connected to friends. Include Breadth-First Search (BFS) to find nearby connections and Depth-First Search (DFS) to explore deeper connection paths. Add inline comments explaining traversal steps, compare recursive and iterative DFS approaches, and suggest practical use cases for BFS versus DFS.

CODE:

```python
#Implement a Python graph using an adjacency list to model a social network where users are connected to fri
from collections import deque
from typing import Dict, List, Set
class SocialNetwork:
    """Graph representation of a social network using adjacency list."""
    def __init__(self):
        self.graph: Dict[str, List[str]] = {}
    def add_user(self, user: str) -> None:
        """Add a user to the network."""
        if user not in self.graph:
            self.graph[user] = []
    def add_connection(self, user1: str, user2: str) -> None:
        """Add a bidirectional friendship connection."""
        self.graph.setdefault(user1, []).append(user2)
        self.graph.setdefault(user2, []).append(user1)
    def bfs(self, start: str) -> List[str]:
        """Breadth-First Search to find nearby connections."""
        visited: Set[str] = set[str]()
        queue: deque[str] = deque[str]([start])
        result: List[str] = []
        while queue:
            user = queue.popleft()  # Explore the front of the queue
            if user not in visited:
                visited.add(user)
                result.append(user)
                # Add all unvisited friends to the queue
                for friend in self.graph.get(user, []):
                    if friend not in visited:
                        queue.append(friend)
        return result
    def dfs_recursive(self, start: str) -> List[str]:
        """Depth-First Search (recursive) to explore deep connections."""
        visited: Set[str] = set[str]()
```

```python
class SocialNetwork:
    def dfs_recursive(self, start: str) -> List[str]:
        result: List[str] = []
        def dfs(user: str):
            if user not in visited:
                visited.add(user)
                result.append(user)
                for friend in self.graph.get(user, []):
                    dfs(friend)
        dfs(start)
        return result
    def dfs_iterative(self, start: str) -> List[str]:
        """Depth-First Search (iterative using stack)."""
        visited: Set[str] = set[str]()
        stack: List[str] = [start]
        result: List[str] = []
        while stack:
            user = stack.pop()  # Explore last added node
            if user not in visited:
                visited.add(user)
                result.append(user)
                # Add friends to stack (reversed for consistent order)
                for friend in reversed[str](self.graph.get(user, [])):
                    if friend not in visited:
                        stack.append(friend)
        return result
# ----------------- Test Function -----------------
def test_social_network():
    sn = SocialNetwork()
    users = ["Alice", "Bob", "Charlie", "David", "Eve"]
    for user in users:
        sn.add_user(user)
    connections = [("Alice", "Bob"), ("Alice", "Charlie"), ("Bob", "David"), ("Charlie", "Eve")]
```

```python
def test_social_network():
    for u1, u2 in connections:
        sn.add_connection(u1, u2)
    print("BFS from Alice:", sn.bfs("Alice"))
    print("DFS Recursive from Alice:", sn.dfs_recursive("Alice"))
    print("DFS Iterative from Alice:", sn.dfs_iterative("Alice"))
test_social_network()
```

OUTPUT:

```
Problems    Output    Debug Console    Terminal    Ports
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:\Users\Pranitha Rao\OneDri
ve\Desktop\AI ASSISTED CODING\.venv\Scripts\python.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI AS
SISTED CODING/Lab-13.py"
BFS from Alice: ['Alice', 'Bob', 'Charlie', 'David', 'Eve']
DFS Recursive from Alice: ['Alice', 'Bob', 'David', 'Charlie', 'Eve']
DFS Iterative from Alice: ['Alice', 'Bob', 'David', 'Charlie', 'Eve']
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION:

The graph uses an adjacency list to represent user connections. BFS explores nearby friends level by level, making it ideal for finding shortest paths or friends-of-friends. DFS explores as deep as possible along each path, either recursively or iteratively with a stack, useful for tracing long connection chains or detecting cycles. Recursive DFS is simpler to write, while iterative DFS avoids recursion limits. BFS is practical for recommendations or network reach, whereas DFS is better for in-depth network exploration or backtracking tasks.